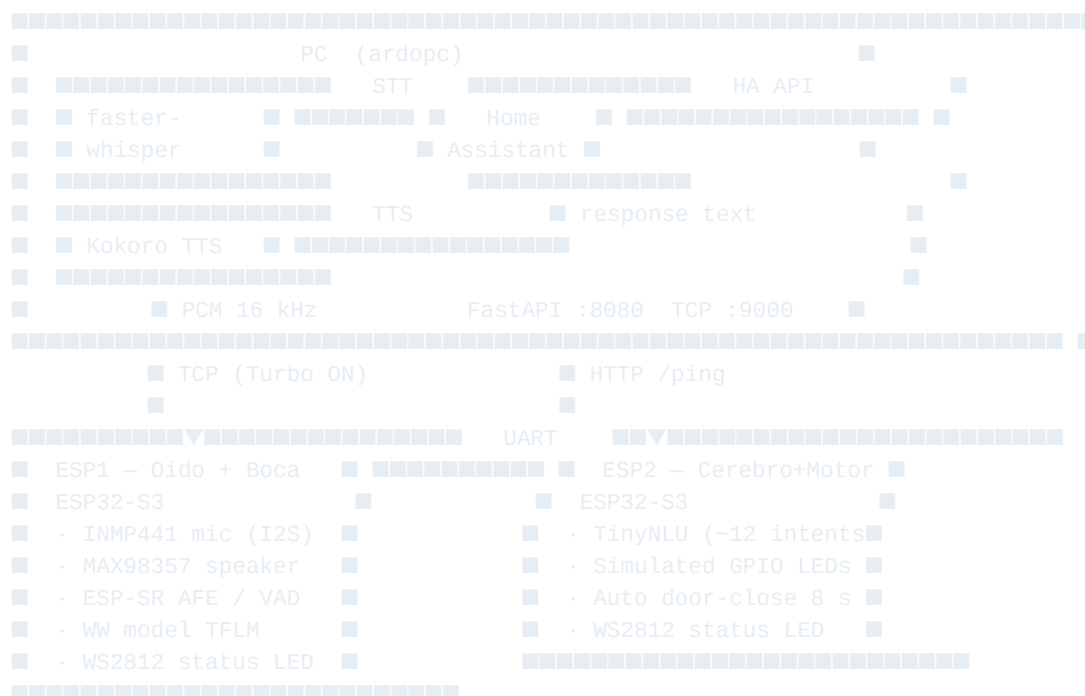


Home Automation Accessibility Assistant

Hackathon project · ESP32-S3 · ESP-IDF · faster-whisper · Kokoro TTS · Home Assistant

System Architecture



Ardo v2 is a voice-controlled home automation assistant designed for wheelchair users and people with motor impairments. A wake-word ('Hey Ardo') triggers either a full cloud-quality Turbo pipeline (speech → faster-whisper STT → Home Assistant NLP → Kokoro TTS) or a lightweight local fallback (energy classification on ESP1 → TinyNLU on ESP2 → simulated actuator response).

Features

Turbo Mode	Wi-Fi TCP stream of raw 16 kHz PCM → faster-whisper on GPU → Home Assistant conversation API -
Local Fallback	5-window RMS energy classifier on ESP1 (6 intent classes) → UART text to ESP2 → TinyNLU keyword
Wake-Word	Custom 'Hey Ardo' model via ESP-SR AFE + TensorFlow Lite Micro with sliding-window probability ave
Home Assistant	Direct integration with HA conversation API at http://ardopc:8123/ . Lights, doors, and robot vacuum are
Two-ESP design	ESP1 handles audio (I2S mic + speaker, Wi-Fi, TCP). ESP2 handles NLU + actuator control. Connecte
Accessibility-first	Hands-free, low-latency, tolerant of noisy environments (AFE with AGC and noise suppression on ESP

Hardware Requirements

Component	Quantity	Notes
ESP32-S3-DevKitC-1	2	8 MB OPI PSRAM, 8/16 MB flash
INMP441 I2S microphone	1	Connected to ESP1
MAX98357A I2S amplifier + speaker	1	Connected to ESP1
PC / laptop with GPU (optional)	1	Runs server, HA VM, faster-whisper
USB-UART cables	2	Programming + debug monitor
LEDs (any colour)	6+	Simulated actuators on ESP2
Resistors 330 Ω	6+	Current limiting for LEDs

Repository Structure

```

ardo_v2/
├── esp1_oido_boca/           # ESP32-S3 – Mic, Wake-Word, Wi-Fi, Speaker
│   ├── platformio.ini
│   ├── partitions.csv       # 8 MB flash layout (model partition for esp-sr)
│   ├── sdkconfig.defaults   # OPI PSRAM, Wi-Fi buffers, FreeRTOS 1000 Hz
│   └── src/
│       ├── CMakeLists.txt
│       ├── idf_component.yml
│       ├── main.cpp          # Full WW + Turbo/Local state machine
│       ├── hey_ardo_model.h   # (you provide – from training)
│       └── hey_ardo_model.cc  # (you provide – from training)
├── esp2_cerebro_motor/      # ESP32-S3 – TinyNLU + simulated actuators
│   ├── platformio.ini
│   └── src/
│       ├── CMakeLists.txt
│       ├── idf_component.yml
│       ├── tiny_nlu.h        # TinyNLU API (12 intents, 5 targets)
│       ├── tiny_nlu.cpp      # Keyword scoring engine + JSON builder
│       └── main.cpp          # UART task + execute_command() + door auto-close
└── server/
    ├── ardo_v2_server.py     # FastAPI :8080 + asyncio TCP :9000
    └── requirements.txt

```

Pin Assignments

ESP1 — Oido + Boca

Signal	GPIO	Notes
I2S MCLK (mic)	2	INMP441
I2S WS / LRCK (mic)	9	INMP441
I2S SD / DATA (mic)	13	INMP441
I2S BCLK (speaker)	4	MAX98357A
I2S LRCK (speaker)	5	MAX98357A
I2S DATA (speaker)	6	MAX98357A
WS2812 RGB LED	48	Onboard DevKitC LED
UART TX → ESP2	17	115200 8N1
UART RX ← ESP2	16	115200 8N1

ESP2 — Cerebro + Motor

Signal	GPIO	Notes
UART RX ← ESP1	16	115200 8N1
UART TX → ESP1	17	115200 8N1
LED: luz principal	1	Simulated actuator
LED: luz recamara	2	Simulated actuator
LED: puerta principal	3	Simulated actuator
LED: puerta trasera	4	Simulated actuator
LED: robot aspiradora	5	Simulated actuator
LED: emergencia	6	Simulated actuator
WS2812 RGB LED	48	Status LED

Communication Protocols

TCP Audio Stream (Turbo Mode)

After wake-word detection and confirming Turbo is ON, ESP1 opens a TCP connection to the PC server on port 9000 and streams raw 16 kHz mono PCM:

```
ESP1 → Server  (binary stream)
[9 bytes]  magic header   "ARDO_AUD1"
[N bytes]  raw int16 LE PCM frames  (AFE-processed, 16 kHz mono)
[9 bytes]  end marker    "ARDO_END1"

Server → ESP1  (binary reply)
[9 bytes]  magic header   "KOKO_AUD1"
[4 bytes]  payload length uint32 LE
[L bytes]  int16 LE PCM  (Kokoro TTS output, resampled to 16 kHz mono)
```

UART Framing (Local Fallback)

```
ESP1 → ESP2  (text line, newline-terminated)
CMD:<transcribed text or intent label>\n
Example:  CMD:enciende la luz\n
```

```
ESP2 → ESP1  (text line, newline-terminated)
RESP:<human-readable response>\n
Example:  RESP:Luz principal encendida\n
```

JSON Command Schema

TinyNLU on ESP2 generates a JSON command that is logged and used internally:

```
{
  "type":      "cmd",
  "source":    "esp2_nlu",
  "priority":  1,           // 1=normal 2=high 3=emergency
  "target":    "light_main",
  "action":    "turn_on",
  "id":        42,
  "params":    {}
}
```

TinyNLU — Supported Intents

Intent	Target	Keywords (sample)
LIGHT_ON	light_main / light_bed	enciende, luz, sala, recamara
LIGHT_OFF	light_main / light_bed	apaga, luz
DOOR_OPEN	door_main / door_back	abre, puerta, principal, trasera
DOOR_CLOSE	door_main / door_back	cierra, puerta

TV_ON	tv	enciende, television, tele
TV_OFF	tv	apaga, television
CURTAIN_OPEN	curtain	abre, cortinas, persiana
CURTAIN_CLOSE	curtain	cierra, cortinas
ROBOT_START	robot_vacuum	aspiradora, limpia, robot
ROBOT_STOP	robot_vacuum	para, detiene, robot
THERMOSTAT	thermostat	temperatura, termostato, grados
EMERGENCY	all	emergencia, ayuda, socorro

Note: EMERGENCY intent always wins — it receives a +2.0 confidence boost regardless of competing intents. Door auto-close triggers 8 seconds after DOOR_OPEN via a temporary FreeRTOS task so the UART loop stays non-blocking.

PC Server

Stack

- FastAPI — REST API on port 8080 (includes /ping endpoint for Turbo detection)
- asyncio TCP server — port 9000, handles ESP1 audio streams
- faster-whisper — GPU-accelerated STT (falls back to CPU automatically)
- Home Assistant conversation API — POST /api/conversation/process (language: es)
- Kokoro TTS — supports both kokoro (KPipeline) and kokoro-onnx backends
- scipy resample_poly — converts Kokoro 24 kHz output to 16 kHz for ESP1 playback

Installation

```
# 1. Install dependencies (choose ONE TTS backend)
pip install -r server/requirements.txt
pip install kokoro          # full quality
# -- OR --
pip install kokoro-onnx     # lighter, ONNX-based

# 2. Set your Home Assistant long-lived access token
export HA_TOKEN='your_token_here'

# 3. Run the server
cd server
python ardo_v2_server.py

# Server exposes:
#   HTTP :8080  — REST API (GET /ping returns {"turbo":true})
#   TCP  :9000  — raw audio stream from ESP1
```

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
httpx==0.27.0
pydantic==2.7.1
faster-whisper==1.0.3
numpy>=1.24.0
scipy>=1.11.0
# TTS — install ONE of:
# kokoro>=0.7.0
# kokoro-onnx>=0.3.4
```

Firmware Build & Flash

Prerequisites

- PlatformIO IDE or CLI
- ESP-IDF toolchain (installed automatically by PlatformIO for espressif32)
- hey_ardo_model.h and hey_ardo_model.cc — your trained wake-word model files, place them in esp1_oido_boca/src/

ESP1

```
cd esp1_oido_boca

# Edit src/main.cpp and set your Wi-Fi credentials and server IP:
#   #define WIFI_SSID    "your_ssid"
#   #define WIFI_PASS    "your_password"
#   #define SERVER_IP    "192.168.x.x"    // PC running ardo_v2_server.py

pio run                # build
pio run -t upload      # flash (port: /dev/ttyUSB0 or adjust)
pio device monitor     # serial monitor at 115200
```

ESP2

```
cd esp2_cerebro_motor
pio run
pio run -t upload      # port: /dev/ttyUSB1 (adjust if needed)
pio device monitor
```

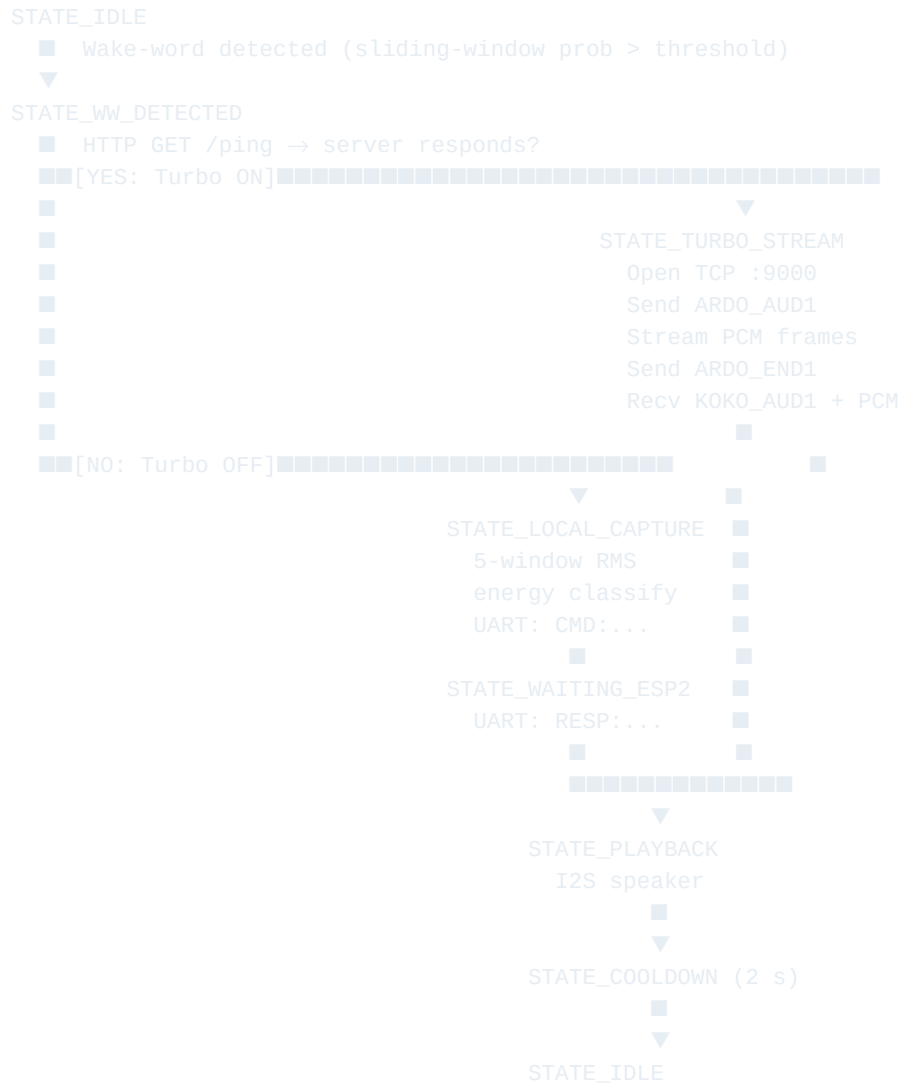
platformio.ini highlights (both ESPs)

```
[env:esp32s3]
platform = espressif32
board    = esp32-s3-devkitc-1
framework = espidf
monitor_speed = 115200
upload_speed = 921600
build_flags =
    -DCORE_DEBUG_LEVEL=3
    -fno-exceptions
```

ESP1 sdkconfig.defaults highlights

```
CONFIG_SPIRAM_MODE_OCT=y          # OPI PSRAM
CONFIG_SPIRAM_SPEED_80M=y
CONFIG_FREERTOS_HZ=1000           # 1 ms tick
CONFIG_LWIP_TCP_SND_BUF_DEFAULT=65535 # larger TCP buffers for audio
CONFIG_LWIP_TCP_WND_DEFAULT=65535
```


ESP1 State Machine



The Turbo check runs every 5 seconds in a background task (`turbo_check_task`) and updates an atomic flag. The `wakeword_task` reads that flag after each detection and branches accordingly using `goto` labels inside the task function to avoid deeply nested conditionals.

Troubleshooting

Symptom	Likely cause	Fix
ESP1 reboots on boot	PSRAM not detected or flash partition verification failed	Verify PSRAM flags in sdkconfig.defaults; check partitions.csv
Wake-word never triggers	Model files missing or wrong tensor architecture	Confirm hey_ardo_model.h is in src/. Increase TENSOR_ARENA_SIZE if needed
Turbo always OFF	Server not reachable from ESP1	Check WIFI_SSID/PASS and SERVER_IP in main.cpp; verify PC firewall allows connections
No audio out of speaker	I2S DMA config or wrong GPIO	Double-check GPIO 4/5/6 wiring to MAX98357A; verify I2S_STD driver initialization
UART not working	Cross-wired TX/RX	ESP1 TX(17) → ESP2 RX(16); ESP1 RX(16) → ESP2 TX(17)
Kokoro fails to load	Backend not installed	pip install kokoro or pip install kokoro-onnx (pick one)
HA returns 401 Unauthorized	Missing or wrong HA_TOKEN env var	Generate a Long-Lived Access Token in HA profile and export HA_TOKEN=token
LEDs on ESP2 don't change	Wrong GPIO or CMakeLists missing 'driver'	Check REQUIRES driver in CMakeLists.txt; verify LED GPIO defs in main.cpp

Contributing & License

This project was built for the Guadalahacks hackathon. Pull requests are welcome. Please open an issue first for major changes. All firmware is under the MIT License unless otherwise noted. The wake-word model (hey_ardo_model.h/.cc) may have separate licensing depending on how it was trained.

Built with love for accessibility · Guadalahacks 2026